parte2

▸ View Source

URL_RANGE = 'https://api.pwnedpasswords.com/range/'

URL_BREACHES = 'https://haveibeenpwned.com/api/v3/breaches'

CABECERAS = {'user-agent': 'practica-paradigmas'}

ERRORES = hide
{400: 'Peticion incorrecta', 404: 'Recurso no encontrado', 429: 'Demasiadas peticiones, espera un momento', 503: 'Servicio no disponible'}

def hash_sha1(contrasena: str) -> str: ▾ View Source

```
17  def hash_sha1(contrasena: str) -> str:
18      """Calcula el hash SHA-1 de una contraseña.
19
20      Args:
21          contrasena (str): Contraseña en texto plano.
22
23      Returns:
24          str: Hash SHA-1 en formato hexadecimal y en mayúsculas.
25      """
26      return hashlib.sha1(contrasena.encode("utf-8")).hexdigest().upper()
```

Calcula el hash SHA-1 de una contraseña.

Args: contrasena (str): Contraseña en texto plano.

Returns: str: Hash SHA-1 en formato hexadecimal y en mayúsculas.

def hash_ntlm(contrasena: str) -> str: ▾ View Source

```
29  def hash_ntlm(contrasena: str) -> str:
30      """Calcula el hash NTLM de una contraseña.
31
32      Args:
33          contrasena (str): Contraseña en texto plano.
34
35      Returns:
36          str: Hash NTLM en formato hexadecimal y en mayúsculas.
37      """
38      return MD4.new(contrasena.encode("utf-16le")).hexdigest().upper()
```

Calcula el hash NTLM de una contraseña.

Args: contrasena (str): Contraseña en texto plano.

Returns: str: Hash NTLM en formato hexadecimal y en mayúsculas.

MODOS_HASH = {'1': (<function hash_ntlm>, '?mode=ntlm'), '2': (<function hash_sha1>, '')}

def mensaje_error(codigo: int) -> str: ▾ View Source

```
47  def mensaje_error(codigo: int) -> str:
48      """Devuelve un mensaje descriptivo según el código de error HTTP recibido.
49
50      Args:
51          codigo (int): Código de estado HTTP.
52
53      Returns:
54          str: Mensaje de error correspondiente.
55      """
56      return ERRORES.get(codigo, f"Error inesperado (codigo {codigo})")
```

Devuelve un mensaje descriptivo según el código de error HTTP recibido.

Args: codigo (int): Código de estado HTTP.

Returns: str: Mensaje de error correspondiente.

def hacer_peticion(url: str, callback_ok, callback_error): ▾ View Source

```
59  def hacer_peticion(url: str, callback_ok, callback_error):
60      """Realiza una petición HTTP GET manejando errores de conexión y códigos de
61
62      Args:
63          url (str): URL de destino.
64          callback_ok (callable): Función a ejecutar si la respuesta es exitosa (
65          callback_error (callable): Función a ejecutar si ocurre un error o fall
66
67      Returns:
68          El resultado de invocar el callback correspondiente.
69      """
70      try:
71          respuesta = requests.get(url, headers=CABECERAS, timeout=10)
72
73      except requests.exceptions.RequestException:
74          return callback_error("No se pudo conectar con el servidor")
75
76      if respuesta.status_code == 200:
77          return callback_ok(respuesta)
78
79      return callback_error(mensaje_error(respuesta.status_code))
```

Realiza una petición HTTP GET manejando errores de conexión y códigos de estado.

Args: url (str): URL de destino. callback_ok (callable): Función a ejecutar si la respuesta es exitosa (código 200). callback_error (callable): Función a ejecutar si ocurre un error o fallo de conexión.

Returns: El resultado de invocar el callback correspondiente.

def obtener_cuenta(texto: str, sufijo: str) -> int: ▾ View Source

```
82  def obtener_cuenta(texto: str, sufijo: str) -> int:
83      """Busca un sufijo de hash en la respuesta y extrae el número de veces filt
84
85      Args:
86          texto (str): Respuesta en texto plano de la API.
87          sufijo (str): Sufijo del hash que se está buscando.
88
89      Returns:
90          int: Número de veces que aparece la contraseña en filtraciones, o 0 si
91      """
92      coincidencias = list(filter(lambda linea: linea.split(":")[0] == sufijo, te
93      return int(coincidencias[0].split(":")[1]) if coincidencias else 0
```

Busca un sufijo de hash en la respuesta y extrae el número de veces filtrada.

Args: texto (str): Respuesta en texto plano de la API. sufijo (str): Sufijo del hash que se está buscando.

Returns: int: Número de veces que aparece la contraseña en filtraciones, o 0 si no se encuentra.

def mensaje_contrasena(cuenta: int) -> str: ▾ View Source

```
96  def mensaje_contrasena(cuenta: int) -> str:
97      """Genera el mensaje correspondiente según el número de filtraciones de un
98
99      Args:
100         cuenta (int): Número de veces que ha aparecido la contraseña.
101
102     Returns:
103         str: Mensaje informativo sobre la filtración.
104     """
105     if cuenta == 0:
106         return "La contrasena no aparece en ninguna filtracion"
107     return f"La contrasena ha aparecido {cuenta} veces en filtraciones"
```

Genera el mensaje correspondiente según el número de filtraciones de una contraseña.

Args: cuenta (int): Número de veces que ha aparecido la contraseña.

Returns: str: Mensaje informativo sobre la filtración.

def comprobar_contrasena(contrasena: str, funcion_hash, parametro: str) -> str:

▾ View Source

```
110  def comprobar_contrasena(contrasena: str, funcion_hash, parametro: str) -> str
111      """Comprueba si una contraseña ha sido filtrada utilizando su hash.
112
113      Args:
114          contrasena (str): Contraseña a comprobar.
115          funcion_hash (callable): Función para calcular el hash (SHA-1 o NTLM).
116          parametro (str): Parámetro adicional para la URL.
117
118      Returns:
119          str: Resultado de la comprobación en forma de mensaje.
120      """
121      hash_completo = funcion_hash(contrasena)
122      prefijo = hash_completo[:5]
123      sufijo = hash_completo[5:]
124
125      return hacer_peticion(
126          f"{URL_RANGE}{prefijo}{parametro}",
127          lambda respuesta: mensaje_contrasena(obtener_cuenta(respuesta.text, su
128          lambda mensaje: f"Error: {mensaje}",
129      )
```

Comprueba si una contraseña ha sido filtrada utilizando su hash.

Args: contrasena (str): Contraseña a comprobar. funcion_hash (callable): Función para calcular el hash (SHA-1 o NTLM). parametro (str): Parámetro adicional para la URL.

Returns: str: Resultado de la comprobación en forma de mensaje.

def mensaje_brechas(brechas: list[dict]) -> str:

▾ View Source

```
132  def mensaje_brechas(brechas: list[dict]) -> str:
133      """Formatea la lista de brechas de seguridad obtenidas para un dominio.
134
135      Args:
136          brechas (list of dict): Lista de diccionarios con la información de la
137
138      Returns:
139          str: Texto formateado con las brechas encontradas o mensaje de que no
140      """
141      if not brechas:
142          return "No hay filtraciones asociadas a este dominio"
143
144      lineas = map(lambda b: f"- {b['Name']} ({b['BreachDate']}): {b['PwnCount']
145      return "\n".join(lineas)
```

Formatea la lista de brechas de seguridad obtenidas para un dominio.

Args: brechas (list of dict): Lista de diccionarios con la información de las filtraciones.

Returns: str: Texto formateado con las brechas encontradas o mensaje de que no hay.

def comprobar_dominio(dominio: str) -> str: ▾ View Source

```
148  def comprobar_dominio(dominio: str) -> str:
149      """Comprueba las brechas de seguridad asociadas a un dominio específico.
150
151      Args:
152          dominio (str): Nombre del dominio a consultar.
153
154      Returns:
155          str: Mensaje con las brechas encontradas o error.
156      """
157      return hacer_peticion(
158          f"{URL_BREACHES}?Domain={dominio}",
159          lambda respuesta: mensaje_brechas(respuesta.json()),
160          lambda mensaje: f"Error: {mensaje}",
161      )
```

Comprueba las brechas de seguridad asociadas a un dominio específico.

Args: dominio (str): Nombre del dominio a consultar.

Returns: str: Mensaje con las brechas encontradas o error.

def modo_contrasenas() -> **None**: ▾ View Source

```
164  def modo_contrasenas() -> None:
165      """Ejecuta el modo interactivo para comprobar contraseñas."""
166      opcion = input("Formato del hash (1: Windows NTLM, 2: estandar SHA1): ")
167
168      if opcion not in MODOS_HASH:
169          print("Opcion no valida")
170          return
171
172      contrasena = input("Contrasena: ")
173      funcion_hash, parametro = MODOS_HASH[opcion]
174
175      print(comprobar_contrasena(contrasena, funcion_hash, parametro))
```

Ejecuta el modo interactivo para comprobar contraseñas.

def modo_dominios() -> **None**: ▾ View Source

```
178  def modo_dominios() -> None:
179      """Ejecuta el modo interactivo para comprobar dominios."""
180      dominio = input("Dominio: ")
181      print(comprobar_dominio(dominio))
```

Ejecuta el modo interactivo para comprobar dominios.

MODOS = {'1': <function modo_contrasenas>, '2': <function modo_dominios>}

def main() -> **None**: ▾ View Source

```
190  def main() -> None:
191      """Punto de entrada principal para la ejecución interactiva del programa."
192      opcion = input("Modo (1: contrasenas, 2: dominios): ")
193
194      if opcion not in MODOS:
195          print("Opcion no valida")
196          return
197
198      MODOS[opcion]()
```

Punto de entrada principal para la ejecución interactiva del programa.

def pruebas() -> **None**: ▾ View Source

```
201  def pruebas() -> None:
202      """Ejecuta una batería de pruebas para verificar el funcionamiento de los 
203      print("hash_sha1")
204      print(hash_sha1("password"))
205      print(hash_sha1("123456"))
206      print(hash_sha1("admin"))
207      print(hash_sha1(""))
208      print(hash_sha1("qwerty"))
209
210      print("\nhash_ntlm")
211      print(hash_ntlm("password"))
212      print(hash_ntlm("123456"))
213      print(hash_ntlm("admin"))
214      print(hash_ntlm(""))
215      print(hash_ntlm("qwerty"))
216
217      print("\nmensaje_error")
218      print(mensaje_error(400))
219      print(mensaje_error(404))
220      print(mensaje_error(429))
221      print(mensaje_error(503))
222      print(mensaje_error(500))
223
224      print("\nobtener_cuenta")
225      texto = "AAA:3\nBBB:10\nCCC:1"
226      print(obtener_cuenta(texto, "AAA"))
227      print(obtener_cuenta(texto, "BBB"))
228      print(obtener_cuenta(texto, "CCC"))
229      print(obtener_cuenta(texto, "DDD"))
230      print(obtener_cuenta("", "AAA"))
231
232      print("\nmensaje_contrasena")
233      print(mensaje_contrasena(0))
```

```
234     print(mensaje_contrasena(1))
235     print(mensaje_contrasena(25))
236     print(mensaje_contrasena(1000))
237     print(mensaje_contrasena(3))
238 
239     print("\nhacer_peticion")
240     print(hacer_peticion(f"{URL_RANGE}5BAA6", lambda r: r.status_code, lambda
241     print(hacer_peticion(f"{URL_RANGE}21BD1", lambda r: r.status_code, lambda
242     print(hacer_peticion(f"{URL_RANGE}ABCDE", lambda r: len(r.text) > 0, lambd
243     print(hacer_peticion(f"{URL_RANGE}XYZ", lambda r: r.status_code, lambda m:
244     print(hacer_peticion("https://dominio-que-no-existe-12345.com", lambda r:
245 
246     print("\ncomprobar_contrasena")
247     print(comprobar_contrasena("password", hash_sha1, ""))
248     print(comprobar_contrasena("123456", hash_sha1, ""))
249     print(comprobar_contrasena("password", hash_ntlm, "?mode=ntlm"))
250     print(comprobar_contrasena("123456", hash_ntlm, "?mode=ntlm"))
251     print(comprobar_contrasena("k8#Vq2!zLp9$wXr7mT", hash_sha1, ""))
252 
253     print("\nmensaje_brechas")
254     print(mensaje_brechas([]))
255     print(mensaje_brechas([{"Name": "Adobe", "BreachDate": "2013-10-04", "PwnC
256     print(mensaje_brechas([{"Name": "A", "BreachDate": "2020-01-01", "PwnCount
257     print(mensaje_brechas([{"Name": "C", "BreachDate": "2019-05-05", "PwnCount
258     print(mensaje_brechas([]))
259 
260     print("\ncomprobar_dominio")
261     print(comprobar_dominio("adobe.com"))
262     print(comprobar_dominio("linkedin.com"))
263     print(comprobar_dominio("dropbox.com"))
264     print(comprobar_dominio("tumblr.com"))
265     print(comprobar_dominio("dominio-que-no-existe-12345.com"))
```

Ejecuta una batería de pruebas para verificar el funcionamiento de los módulos.